PLACEMENT INTERVIEW REVISION

Seaborn Complete Revision Guide

Every plot type, figure-level vs axes-level, diagrams, code and real interview questions.

18 Topics | All Plot Types | Diagrams | Gotchas | 30 Q&A

THE SINGLE MOST IMPORTANT THING IN THIS DOC

Topic 3: Figure-level vs Axes-level functions. This is the #1 Seaborn interview question and it also explains every confusing error you face (why `figsize` does not work, why `ax=` fails, why some functions return a Grid). Read that topic twice.

Revision notes — Statistical Data Visualization with Seaborn · Companion doc: **Matplotlib Complete Revision Guide**

Contents

1 What is Seaborn + Why over Matplotlib
2 Datasets & Long vs Wide data
3 Figure-level vs Axes-level (CORE)
4 Relational: scatterplot, lineplot, relplot
5 Categorical scatter: stripplot, swarmplot
6 boxplot, violinplot, boxenplot
7 barplot, countplot, pointplot
8 Distribution: histplot, kdeplot, ecdf, displot
9 jointplot & JointGrid
10 pairplot & PairGrid
11 heatmap & clustermap
12 Regression: regplot, lmplot, residplot
13 FacetGrid (multiplots)
14 Styling: theme, style, context, palette
15 Which plot when — master table
16 Common errors & gotchas
17 Cheat sheet
18 Real Interview Questions & Answers

▸ The 5 questions interviewers always ask about Seaborn

#	Question	Topic
1	Seaborn vs Matplotlib — difference and when to use which	1
2	Figure-level vs Axes-level functions	3
3	stripplot vs swarmplot / boxplot vs violinplot	5, 6
4	barplot vs countplot (and what the black line means)	7
5	How do you check correlation between all columns?	11

1 What is Seaborn + Why over Matplotlib

Seaborn is a **statistical** data visualization library built **on top of Matplotlib**, and it works directly with Pandas DataFrames.

▸ What it adds on top of Matplotlib

- **Statistics built in** — it computes mean, confidence interval, KDE, regression line for you. Matplotlib only draws what you give it.
- **DataFrame aware** — you pass column *names*: `sns.scatterplot(data=df, x='age', y='fare', hue='sex')`.
- **hue / size / style / col / row** — one keyword splits the plot by a category. In Matplotlib you would write a loop.
- **Better defaults** — nicer colours, grid, spacing, automatic legend.
- **Complex plots in one line** — pairplot, heatmap, violin, jointplot.

IN INTERVIEW, SAY THIS

"Seaborn is not a replacement for Matplotlib, it is a wrapper built on it. Seaborn handles the statistical work and the DataFrame interface, while Matplotlib handles the actual drawing. Every axes-level Seaborn function returns a Matplotlib Axes, so I use Seaborn for the chart and Matplotlib for fine-tuning title, limits and annotations."

Point	Matplotlib	Seaborn
Level	Low level, full control	High level, fewer lines
Data input	Arrays / lists	DataFrame + column names
Statistics	You compute yourself	Built in (mean, CI, KDE, regression)
Grouping by category	Manual loop	`hue`, `col`, `row`, `style`, `size`
Default look	Plain	Attractive themes
Best for	Custom and precise charts	Fast EDA and statistical charts

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

sns.set_theme()               # turn on seaborn look
print(sns.__version__)
```

2 Datasets & Long vs Wide Data

▸ Built-in practice datasets

```
sns.get_dataset_names()
tips    = sns.load_dataset('tips')      # restaurant bills - most used
titanic = sns.load_dataset('titanic')
iris    = sns.load_dataset('iris')
flights = sns.load_dataset('flights')   # time series, good for heatmap
```

▸ Long (tidy) vs Wide format — important concept

Seaborn mostly wants data in **long / tidy** format: **one row = one observation**, and every variable has its own column.

WIDE format (spreadsheet style)

month	2023	2024
Jan	100	140
Feb	120	160

Year is hidden in the column NAMES → hard for hue=

df.melt()

LONG / TIDY format (seaborn style)

month	year	sales
Jan	2023	100
Jan	2024	140
Feb	2023	120
Feb	2024	160

Now: x='month', y='sales', hue='year' just works

Rule: **one row = one observation**, one column = one variable. Convert with df.melt(); reverse with df.pivot().

Figure 2.1 — Long format is what Seaborn's `hue`, `col` and `row` need.

```
long_df = wide_df.melt(id_vars='month', var_name='year', value_name='sales')
sns.lineplot(data=long_df, x='month', y='sales', hue='year')
```

3 Figure-level vs Axes-level Functions (CORE TOPIC)

Seaborn has two kinds of functions. Knowing the difference answers half the Seaborn interview.

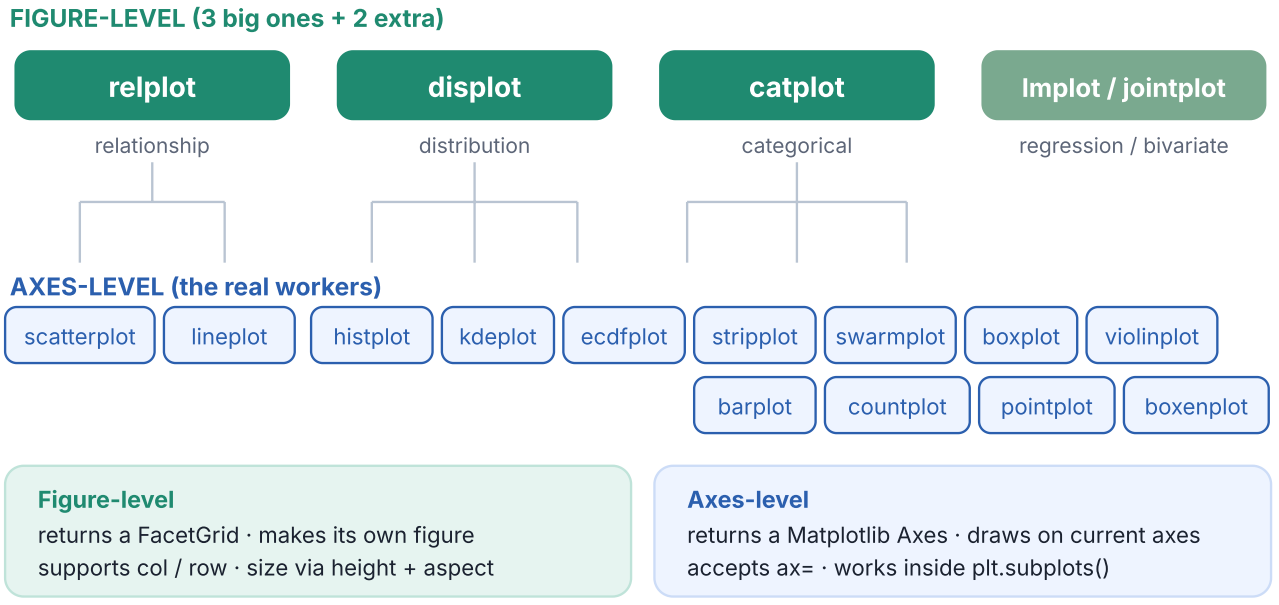


Figure 3.1 — The Seaborn function map. Each figure-level function is a wrapper over its axes-level family, selected by `kind=`.

▸ The 3 families (remember this table — it organises all of Seaborn)

Figure-level	Family	Axes-level functions it can call (`kind=`)
`relplot`	Relational	`scatterplot` (default), `lineplot`
`displot`	Distribution	`histplot` (default), `kdeplot`, `ecdfplot` (+ `rugplot`)
`catplot`	Categorical	`stripplot` (default), `swarmplot`, `boxplot`, `violinplot`, `boxenplot`, `barplot`, `countplot`, `pointplot`
`lmplot`	Regression	`regplot`
`jointplot` / `pairplot`	Grid plots	return `JointGrid` / `PairGrid`

▸ Same chart, two ways

```
# axes-level
fig, ax = plt.subplots(figsize=(8,5))
sns.scatterplot(data=tips, x='total_bill', y='tip', hue='sex', ax=ax)

# figure-level - same chart + can split into panels
sns.relplot(data=tips, x='total_bill', y='tip', hue='sex',
            kind='scatter', col='time', height=5, aspect=1.2)
```

Point	Figure-level	Axes-level
Returns	`FacetGrid` object (`g`)	Matplotlib `Axes`
Creates figure?	Yes, its own figure — it controls everything	No, draws on the current/given axes
`ax=` parameter	**Not allowed** (TypeError)	Allowed
Size control	`height` + `aspect` (per panel)	`figsize` in `plt.subplots()`
`col` / `row` faceting	Yes — main advantage	No
Legend	Outside the plot, automatic	Inside the plot
Use inside subplots	No	Yes — this is why axes-level exists
Customizing	`g.set_axis_labels()`, `g.fig`, `g.axes`	Normal matplotlib `ax.set_*()`

TWO ERRORS THIS EXPLAINS

1. `sns.displot(..., ax=ax)` → error, because figure-level functions make their own figure. Use `sns.histplot(..., ax=ax)` instead.
2. `plt.figure(figsize=(12,6))` before a `catplot` does nothing — catplot creates a new figure. Use `height=` and `aspect=`.

IN INTERVIEW, SAY THIS

"Axes-level functions like `scatterplot` or `boxplot` draw on a single Matplotlib Axes and accept `ax=`, so they fit inside my own `plt.subplots()` grid. Figure-level functions like `relplot`, `displot` and `catplot` create and own the whole figure, return a FacetGrid, and support `col` and `row` to split data into panels. So: one panel inside my layout → axes-level; many panels split by a category → figure-level."

4 Relational Plots: scatterplot, lineplot, relplot

"Relational" = relationship between **two numerical** columns.

▸ scatterplot

```
sns.scatterplot(data=tips, x='total_bill', y='tip',
                hue='sex',        # colour by category
                style='time',     # marker shape by category
                size='size',      # dot size by a column
                sizes=(20,200),   # min and max dot size
                palette='Set2', alpha=0.8)
```

4 encoding channels: x, y, `hue` (colour), `style` (shape), `size`. Using all at once makes it unreadable — use 3 at most.

▸ lineplot

```
sns.lineplot(data=flights, x='year', y='passengers', hue='month')
```

MOST ASKED LINEPLOT QUESTION: "WHAT IS THE SHADED BAND?"

If the same x value appears many times, `lineplot` automatically plots the **mean** of y and shades the **95% confidence interval** around it. Turn it off with `errorbar=None` (older versions: `ci=None`), or change the summary with `estimator='median'` / `estimator=np.sum`.

```
sns.lineplot(data=df, x='day', y='sales', estimator='mean', errorbar=('ci',95))
sns.lineplot(data=df, x='day', y='sales', errorbar='sd')     # std dev band
sns.lineplot(data=df, x='day', y='sales', errorbar=None)     # no band, faster
sns.lineplot(data=df, x='day', y='sales', markers=True, dashes=False)
```

▸ relplot — the figure-level version

```
sns.relplot(data=tips, x='total_bill', y='tip',
            kind='scatter',          # or 'line'
            hue='sex', col='time', row='smoker',
            height=4, aspect=1.2)
```

`col='time'` makes one panel per value of `time`. `col_wrap=3` wraps panels into rows when there are many categories.

5 Categorical Scatter: stripplot & swarmplot

Use when x is **categorical** and y is **numerical**, and you want to see **every single data point**.

▸ stripplot

```
sns.stripplot(data=tips, x='day', y='total_bill',
              jitter=0.25,      # spread points sideways; jitter=False stacks them in a line
              hue='sex', dodge=True,   # split hue into separate columns
              size=5, alpha=0.7)
```

Points are placed randomly left/right (jitter) so they do not sit exactly on top of each other. **Points can still overlap.**

▸ swarmplot

```
sns.swarmplot(data=tips, x='day', y='total_bill', hue='sex', dodge=True)
```

Uses an algorithm to shift points so that **no two points overlap**. The shape it makes (like a beeswarm) shows the density — wider part = more values there.

Point	stripplot	swarmplot
Point placement	Random jitter	Computed, non-overlapping
Overlap	Possible	None
Speed	Fast	Slow (heavy algorithm)
Big data	Works fine	Gives a warning: "points cannot be placed"
Shows density	Roughly	Clearly, by the width of the swarm

BEST PRACTICAL ANSWER

"Both show individual observations for each category. `stripplot` only adds random jitter so points can still overlap, while `swarmplot` positions every point so none overlap, which shows density better but is slow on large data. In practice I overlay one of them on a boxplot or violinplot — the box gives the summary and the points show the actual sample size."

```
sns.boxplot(data=tips, x='day', y='total_bill', color='lightgrey')
sns.stripplot(data=tips, x='day', y='total_bill', color='black', size=3, alpha=.5)
```

6 boxplot, violinplot, boxenplot

These compare the **distribution** of a numerical column across categories.

▸ boxplot

```
sns.boxplot(data=tips, x='day', y='total_bill',
            hue='sex',          # side-by-side boxes per category
            orient='h',         # horizontal
            showfliers=False,   # hide outlier dots
            width=0.6, palette='Set2')
```

Shows Q1, median, Q3, whiskers at 1.5×IQR, and outliers as separate dots. (Full anatomy diagram is in the Matplotlib doc, Topic 9.)

▸ violinplot

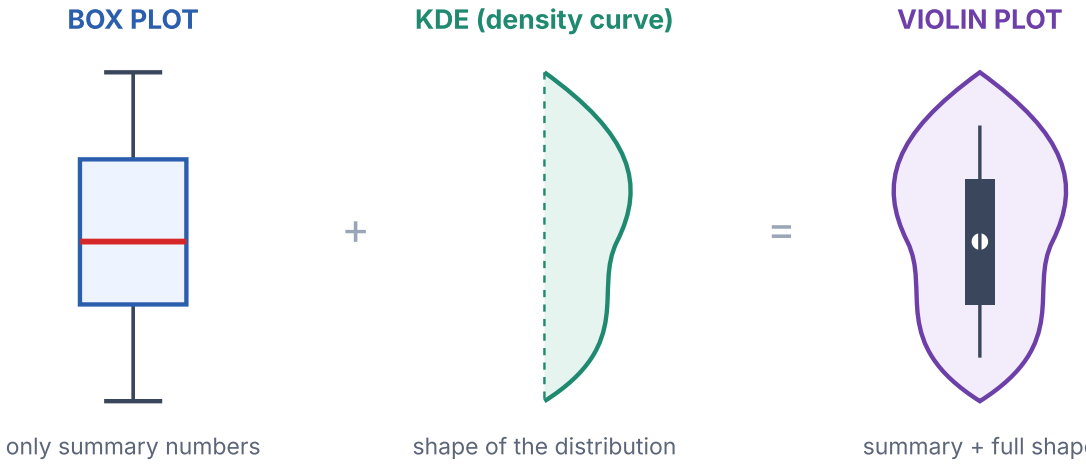


Figure 6.1 — A violin plot is a mirrored KDE curve with a mini box plot inside.

```
sns.violinplot(data=tips, x='day', y='total_bill',
               hue='sex', split=True,     # 2 categories: half violin each - very useful
               inner='quartile',          # 'box'(default), 'quartile', 'point', 'stick',
None
               bw_adjust=0.8, cut=0)      # cut=0 stops the curve at real data range
```

Point	boxplot	violinplot
Shows	5-number summary + outliers	Full distribution shape (KDE) + summary inside
Bimodal data (2 peaks)	**Cannot show it** — looks normal	Shows both peaks clearly
Outliers	Marked as dots	Not marked separately
Small sample	Honest	KDE can invent a smooth shape that is not real
Audience	Everyone understands	Needs some explanation

▸ boxenplot (letter-value plot)

```
sns.boxenplot(data=df, x='class', y='fare')
```

Like a boxplot but draws more quantile boxes. Made for **large datasets**, where it shows the tails far better than a normal boxplot.

7 barplot, countplot, pointplot

▸ barplot — shows an AGGREGATE

```
sns.barplot(data=tips, x='day', y='total_bill',
            estimator='mean',          # default = mean. Can be 'median', 'sum', np.std
            errorbar=('ci', 95),       # black line on top (old name: ci=95)
            hue='sex', palette='pastel')
```

VERY COMMON QUESTION: WHAT IS THE BLACK LINE ON THE BAR?

It is the **error bar — the 95% confidence interval of the mean**, computed by bootstrapping. A long line means that group has few samples or high variance, so the mean is not reliable. Remove it with `errorbar=None`.

▸ countplot — counts rows

```
sns.countplot(data=titanic, x='class', hue='survived')
```

Only takes **one** categorical column and plots its frequency. It is the categorical version of a histogram.

Point	barplot	countplot
Inputs	x (category) **and** y (numerical)	only x **or** y (category)
Bar height	Mean (or chosen estimator) of y	Count of rows
Equivalent to	`df.groupby(x)[y].mean()`	`df[x].value_counts()`
Error bar	Yes (CI)	No

So `countplot(x='day')` is exactly `barplot(x='day', y=<count>, estimator='count')`. Remember: **countplot = barplot with counting built in.**

▸ pointplot

```
sns.pointplot(data=tips, x='day', y='total_bill', hue='sex', dodge=True, markers=['o','s'])
```

Shows the same mean + CI as a barplot, but as a **point joined by a line**. The slope of the line makes it easy to see **interaction** between two categories — if the lines are not parallel, the effect of one category depends on the other. Preferred over bars when comparing trends across groups.

▸ catplot — figure-level wrapper for all of the above

```
sns.catplot(data=tips, x='day', y='total_bill', kind='box',  col='sex', height=4)
# kind = strip | swarm | box | violin | boxen | bar | count | point
```

8 Distribution Plots: histplot, kdeplot, ecdfplot, displot

▸ histplot

```
sns.histplot(data=tips, x='total_bill',
             bins=30,            # or binwidth=5
             kde=True,           # overlay the smooth KDE curve
             hue='sex',
             multiple='stack',   # 'layer'(default) | 'stack' | 'dodge' | 'fill'
             stat='density',     # 'count'(default) | 'density' | 'probability' | 'percent'
             element='step')
```

Giving both `x` and `y` makes a **2D histogram** (heatmap of counts) — good for large scatter data.

▸ kdeplot — Kernel Density Estimate

```
sns.kdeplot(data=tips, x='total_bill', hue='sex', fill=True, alpha=.4)
sns.kdeplot(data=tips, x='total_bill', y='tip')      # 2D contour plot
sns.kdeplot(data=tips, x='total_bill', bw_adjust=0.5, cut=0)
```

WHAT IS KDE IN SIMPLE WORDS?

It is a **smooth histogram**. Instead of putting values into bins, it places a small curve (kernel, usually Gaussian) on every data point and adds them all up. The result is a smooth curve whose **total area is 1**, so the y-axis is density, not count. `bw` (bandwidth) controls smoothness: small bw = spiky, large bw = over-smooth. It is the "bins" problem in a different form.

▸ ecdfplot — cumulative

```
sns.ecdfplot(data=tips, x='total_bill', hue='sex')
```

For every value it shows what fraction of data is below it. No bins, no smoothing — so nothing is hidden. Good answer for "how do you compare two distributions without bin bias?"

▸ rugplot

```
sns.kdeplot(data=tips, x='total_bill')
sns.rugplot(data=tips, x='total_bill', height=.05)   # small ticks for each observation
```

▸ displot — figure-level

```
sns.displot(data=tips, x='total_bill', kind='hist', kde=True, col='sex', height=4)
# kind = 'hist' (default) | 'kde' | 'ecdf'
```

DISTPLOT IS REMOVED — KNOW THIS

Old code used `sns.distplot()` (histogram + KDE together). It is **deprecated and removed in Seaborn 0.14**. Replace it with:

- `sns.histplot(x, kde=True)` — axes-level
- `sns.displot(x, kde=True)` — figure-level

If an interviewer mentions `distplot`, saying this scores points.

9 jointplot & JointGrid

A **bivariate** plot in the centre plus the **distribution of each variable** on the sides. One chart that answers both "how are they related?" and "how is each one distributed?"

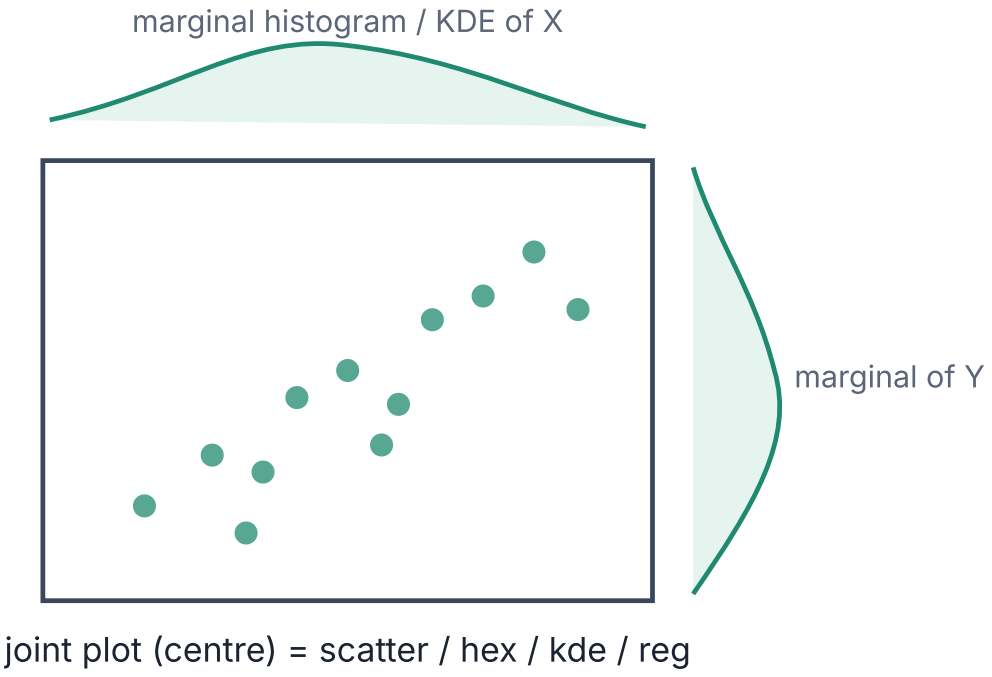


Figure 9.1 — jointplot = joint (centre) + marginal (side) distributions.

```
sns.jointplot(data=tips, x='total_bill', y='tip',
              kind='scatter',       # 'scatter'|'kde'|'hex'|'reg'|'resid'|'hist'
              hue='sex',
              marginal_kws=dict(bins=25, fill=True),
              height=6, ratio=4)    # ratio = centre size vs side size
```

kind	Use when
`scatter`	Normal size data
`hex`	Very large data — hexagon density instead of overlapping dots
`kde`	Smooth 2D contour, shows density "hills"
`reg`	Scatter + regression line + marginal histograms

JointGrid is the manual version — you decide each part yourself:

```
g = sns.JointGrid(data=tips, x='total_bill', y='tip')
g.plot_joint(sns.scatterplot)
g.plot_marginals(sns.boxplot)
```

`jointplot` is figure-level and returns a `JointGrid`, so `ax=` is not allowed.

10 pairplot & PairGrid

Plots **every numerical column against every other** — the fastest way to explore a new dataset.

```
sns.pairplot(iris,
             hue='species',         # colour by category - also changes diagonal to KDE
             diag_kind='kde',       # 'auto' | 'hist' | 'kde' | None
             kind='scatter',        # 'scatter' | 'kde' | 'hist' | 'reg'
             vars=['sepal_length','petal_length'],   # only these columns
             corner=True,           # draw only the lower triangle (no duplicates)
             height=2.5, palette='Set1')
```

▸ How to read it

- **Diagonal** = distribution of that single column (histogram or KDE).
- **Off-diagonal** = scatter plot of the column pair — shows correlation, clusters and outliers.
- It is **symmetric**: upper and lower triangles show the same pairs mirrored, which is why `corner=True` is useful.

LIMITATION TO MENTION

pairplot plots n×n charts. With 20 numerical columns that is 400 small plots — unreadable and slow. For many columns use a **correlation heatmap** first, then pairplot only the few interesting columns using `vars=`.

```
g = sns.PairGrid(iris, hue='species')     # full manual control
g.map_upper(sns.scatterplot)
g.map_lower(sns.kdeplot)
g.map_diag(sns.histplot)
g.add_legend()
```

11 heatmap & clustermap

Matrix plots. Input must be a **2D matrix / pivot table**, not raw long data.

▸ heatmap

```
sns.heatmap(df.corr(numeric_only=True),
            annot=True,        # write the number in each cell
            fmt='.2f',         # number format ('d' for integers)
            cmap='coolwarm',   # diverging, because corr goes -1 to +1
            vmin=-1, vmax=1, center=0,
            linewidths=0.5, linecolor='white',
            cbar=True, square=True,
            mask=np.triu(np.ones_like(df.corr(), dtype=bool)))  # hide upper triangle
```

▸ Two main uses

1. **Correlation matrix** — `sns.heatmap(df.corr(), annot=True)`. Find which features relate to the target, and spot multicollinearity.
2. **Pivot table** — two categories vs one value:

```
pt = flights.pivot(index='month', columns='year', values='passengers')
sns.heatmap(pt, cmap='YlGnBu')
```

INTERVIEW POINTS THAT SOUND PROFESSIONAL

- Use a **diverging** cmap (`coolwarm` , `RdBu`) for correlation because 0 is a meaningful centre; use a **sequential** cmap for counts.
- Always set `vmin=-1, vmax=1` for correlation so colours are comparable across datasets.
- Use `mask` to hide the upper triangle — a correlation matrix is symmetric, so half of it is duplicate.
- `df.corr()` gives **Pearson** (linear) by default. `method='spearman'` gives rank correlation, which catches non-linear but monotonic relations.

▸ clustermap

```
sns.clustermap(iris.drop(columns='species'),
               cmap='viridis',
               standard_scale=1,     # scale each column 0-1 (1=columns, 0=rows)
               z_score=0,            # or standardize instead
               method='average', metric='euclidean',
               row_cluster=True, col_cluster=True,
               figsize=(8,8))
```

Point	heatmap	clustermap
Row/column order	As given in the data	**Reordered** so similar rows/columns sit together
Extra element	—	**Dendrogram** (tree) on top and left
Method behind it	Just colouring	**Hierarchical clustering**
Level	Axes-level (`ax=` works)	Figure-level (`ax=` does **not** work)
Scaling	Do it yourself	`standard_scale` / `z_score` built in

Why scaling matters in clustermap: clustering uses distance, so a column measured in thousands would dominate one measured in decimals. That is why you normalize first.

12 Regression Plots: regplot, lmplot, residplot

```
sns.regplot(data=tips, x='total_bill', y='tip',
            ci=95, order=2,          # order=2 -> polynomial fit
            lowess=True,             # non-parametric smooth curve
            scatter_kws={'alpha':.4}, line_kws={'color':'red'})

sns.lmplot(data=tips, x='total_bill', y='tip',
           hue='smoker', col='time', height=4)      # figure-level version

sns.residplot(data=tips, x='total_bill', y='tip')
```

Function	Level	Notes
`regplot`	Axes-level	Scatter + fitted line + CI band. Accepts `ax=`.
`lmplot`	Figure-level	Same chart but supports `hue`, `col`, `row`.
`residplot`	Axes-level	Plots the residuals (errors). If a pattern is visible, a linear model is not suitable.

Interview link to ML: "Residual plot should look like random noise around zero. A curve or funnel shape means the linear assumption or constant-variance assumption is broken."

13 FacetGrid (Multiplots)

Faceting = splitting one chart into many small panels by a category. Also called "small multiples". This is what `col` and `row` do internally.

```
g = sns.FacetGrid(tips, col='time', row='smoker', hue='sex',
                  height=3.5, aspect=1.2, margin_titles=True)
g.map(sns.scatterplot, 'total_bill', 'tip')      # or g.map_dataframe(...)
g.add_legend()
g.set_axis_labels('Bill', 'Tip')
g.set_titles(col_template='{col_name}')
g.fig.suptitle('Tips analysis', y=1.03)
```

Parameter	Meaning
`col` / `row`	Category that creates columns / rows of panels
`col_wrap=3`	Wrap panels into new rows after 3 (use only with `col`, not `row`)
`height`, `aspect`	Height of each panel in inches; width = height × aspect
`sharex` / `sharey`	Default True — keeps all panels on the same scale for fair comparison
`g.axes`, `g.fig`	Access the underlying Matplotlib objects for customization

Three Grid classes to remember: **FacetGrid** (same plot repeated per category), **PairGrid** (all column pairs), **JointGrid** (joint + marginals).

WHY FACET INSTEAD OF HUE?

" `hue` puts everything in one panel, which is better for direct comparison but becomes cluttered with many categories or overlapping points. Faceting gives each category its own clean panel, and since the axes are shared, comparison is still fair."

14 Styling: theme, style, context, palette

▸ One line that sets everything

```
sns.set_theme(style='whitegrid', context='talk', palette='Set2', font_scale=1.1)
```

▸ style — background & grid (5 options)

Style	Look
`darkgrid`	Default. Grey background + white grid
`whitegrid`	White background + grey grid — best for reports
`dark` / `white`	No grid
`ticks`	White + tick marks on the axes — best for publications

▸ context — size of fonts, lines, markers (4 options)

`paper` < `notebook` (default) < `talk` < `poster`. Use `talk` or `poster` for presentation slides.

```
sns.set_context('talk', font_scale=1.2)
sns.despine(left=True)          # remove top & right spines (and left here)
sns.reset_defaults()            # back to matplotlib defaults
```

▸ Palettes — 3 types (same logic as Matplotlib colormaps)

Type	Use for	Examples
Qualitative	Unordered categories	`deep`, `muted`, `pastel`, `bright`, `dark`, `colorblind`, `Set1`, `tab10`
Sequential	Low → high values	`Blues`, `rocket`, `mako`, `viridis`, `YlGnBu`
Diverging	Meaningful midpoint (0)	`coolwarm`, `vlag`, `icefire`, `RdBu`

```
sns.color_palette('Set2')              # get a palette
sns.set_palette('pastel')              # set globally
sns.color_palette('husl', 8)           # 8 evenly spaced distinct colours
sns.light_palette('seagreen', as_cmap=True)
sns.palplot(sns.color_palette('deep')) # preview the colours
```

▸ Size control — the classic confusion

```
# AXES-LEVEL: use matplotlib figsize
plt.figure(figsize=(10,5))
sns.boxplot(data=tips, x='day', y='total_bill')

# FIGURE-LEVEL: figsize is IGNORED, use height + aspect
sns.catplot(data=tips, x='day', y='total_bill', kind='box', height=5, aspect=2)
```

15 Which Plot When — Master Table

Data situation	Plot	Function
1 numerical	Distribution	`histplot` , `kdeplot` , `ecdfplot` , `boxplot`
1 categorical	Frequency	`countplot`
2 numerical	Relationship	`scatterplot` , `regplot` , `jointplot`
2 numerical + time order	Trend	`lineplot`
1 cat + 1 num (summary)	Compare averages	`barplot` , `pointplot`
1 cat + 1 num (spread)	Compare distributions	`boxplot` , `violinplot` , `boxenplot`
1 cat + 1 num (all points)	Show every observation	`stripplot` , `swarmplot`
2 categorical	Cross counts	`countplot(hue=)` , `heatmap(crosstab)`
Many numerical	All relations at once	`pairplot` , `heatmap(df.corr())`
Matrix / pivot table	Colour grid	`heatmap` , `clustermap`
Add a category to any plot	Split	`hue` , or `col` / `row` faceting

EDA ORDER THAT INTERVIEWERS LIKE TO HEAR

"First univariate — `histplot` for numerical columns and `countplot` for categorical ones. Then bivariate — `scatterplot` for numerical pairs, `boxplot` for category vs numerical, and a correlation `heatmap` for the whole table. Then multivariate — add `hue` or facet with `col` , and `pairplot` on the important columns."

16 Common Errors & Gotchas

Problem	Reason & fix
`figsize` has no effect	You used a figure-level function. Use `height` + `aspect`.
`TypeError: got an unexpected keyword argument 'ax'`	Figure-level functions do not accept `ax=`. Use the axes-level version (`histplot` instead of `displot`).
Seaborn plot is empty inside `plt.subplots()`	Pass `ax=ax[0,0]` to the Seaborn function.
swarmplot warning: "x% of points cannot be placed"	Too many points for the space. Increase figure width, reduce `size=`, or use `stripplot`.
`distplot` not found	Removed in new versions. Use `histplot(kde=True)` or `displot`.
`FutureWarning: ci is deprecated`	New name is `errorbar=('ci',95)` or `errorbar=None`.
Palette warning when using `palette` without `hue`	Newer Seaborn wants `hue=x, legend=False` when colouring by the x category.
heatmap shows nothing / error on strings	`df.corr(numeric_only=True)` — text columns must be excluded.
Heatmap labels cut off	`plt.tight_layout()` or `plt.yticks(rotation=0)`.
`hue` does not work on my data	Data is in wide format. Convert with `df.melt()` to long format first.
Seaborn style changed my Matplotlib charts	`sns.set_theme()` changes global rcParams. Reset with `sns.reset_defaults()`.

17 Cheat Sheet

Setup

```
import seaborn as sns
import matplotlib.pyplot as plt
sns.set_theme(style='whitegrid')
tips = sns.load_dataset('tips')
```

Relational

```
sns.scatterplot(data=d,x=,y=,hue=,size=,style=)
sns.lineplot(data=d,x=,y=,hue=,errorbar=None)
sns.relplot(..., kind='line', col=, row=)
```

Matrix

```
sns.heatmap(df.corr(numeric_only=True),
   annot=True, fmt='.2f', cmap='coolwarm',
   vmin=-1, vmax=1, linewidths=.5, mask=m)
sns.clustermap(df, standard_scale=1,
   cmap='viridis')   # + dendrogram
```

Grids

```
sns.jointplot(x=,y=,kind='hex'|'kde'|'reg')
sns.pairplot(df,hue=,corner=True,diag_kind='kde')
g = sns.FacetGrid(df,col=,row=,hue=)
g.map(sns.scatterplot,'x','y'); g.add_legend()
```

Categorical

```
sns.stripplot(x=,y=,jitter=.25,dodge=True)
sns.swarmplot(x=,y=)        # no overlap, slow
sns.boxplot(x=,y=,hue=,showfliers=False)
sns.violinplot(x=,y=,hue=,split=True,inner='quart
sns.boxenplot(x=,y=)        # large data
sns.barplot(x=,y=,estimator='mean',errorbar=
('ci',95))
sns.countplot(x=)           # value_counts
sns.pointplot(x=,y=,hue=,dodge=True)
sns.catplot(..., kind='box', col=)
```

Distribution

```
sns.histplot(x=,bins=30,kde=True,
             multiple='stack',stat='density')
sns.kdeplot(x=,hue=,fill=True,bw_adjust=.5)
sns.ecdfplot(x=)
sns.rugplot(x=)
sns.displot(x=, kind='kde', col=)
```

Regression

```
sns.regplot(x=,y=,order=2,lowess=True)
sns.lmplot(x=,y=,hue=,col=)
sns.residplot(x=,y=)
```

Style

```
sns.set_theme(style=,context=,palette=)
style: darkgrid|whitegrid|dark|white|ticks
context: paper|notebook|talk|poster
sns.despine(); sns.set_palette('Set2')
sns.color_palette('husl', 8)
```

Figure vs Axes level

```
FIGURE: relplot displot catplot
        lmplot jointplot pairplot
        -> FacetGrid, height+aspect, col/row
AXES  : scatter/line/hist/kde/box/violin/
        bar/count/strip/swarm/heatmap/regplot
        -> Axes, ax=, figsize
```

18 Real Interview Questions & Answers

Short, speakable answers — the way you can actually say them in the interview.

▸ A. Basics

1. What is Seaborn and how is it different from Matplotlib?

Seaborn is a statistical visualization library built on top of Matplotlib. The difference is that Seaborn works directly with DataFrames and does the statistics itself — mean, confidence interval, KDE, regression — and gives complex plots in one line. Matplotlib is lower level and only draws exactly what I give it. I use Seaborn for the chart and Matplotlib to fine-tune it.

2. Is Seaborn a replacement for Matplotlib?

No. Seaborn is a wrapper over Matplotlib. Every axes-level Seaborn function returns a Matplotlib Axes, so both are used together.

3. What is figure-level and axes-level in Seaborn?

Axes-level functions like `scatterplot`, `boxplot`, `histplot` draw on one Matplotlib Axes and accept `ax=`, so they work inside my own subplot grid. Figure-level functions like `relplot`, `displot`, `catplot` create their own figure, return a FacetGrid and support `col` and `row` to split data into panels. Figure-level does not accept `ax=`, and its size is set with `height` and `aspect`, not `figsize`.

4. Name the three main figure-level functions and what they wrap.

`relplot` wraps scatterplot and lineplot. `displot` wraps histplot, kdeplot and ecdfplot. `catplot` wraps all categorical plots — strip, swarm, box, violin, boxen, bar, count, point. You choose with `kind=`.

5. What is long (tidy) format and why does Seaborn need it?

Long format means one row is one observation and each variable has its own column. Seaborn needs it because `hue`, `col` and `row` take a column name — if a variable is hidden inside column headers (wide format), Seaborn cannot use it. I convert with `df.melt()`.

6. What does hue do?

`hue` splits the plot by a categorical column and gives each group a different colour, with an automatic legend. In Matplotlib I would have to do a groupby and loop manually.

▸ B. Plot comparisons (asked most)

7. stripplot vs swarmplot?

Both show every individual point for each category. `stripplot` only adds random jitter, so points can still overlap. `swarmplot` calculates positions so that no points overlap, which shows density better but is slow and warns on large data. For big datasets I use stripplot with low alpha.

8. boxplot vs violinplot?

Boxplot shows the five-number summary and outliers. Violin plot shows the full distribution shape using a mirrored KDE, with a small box inside. The big difference is that a boxplot cannot show a bimodal distribution — two peaks look like one normal group — while the violin shows both peaks. But on small samples the violin's KDE can create a smooth shape that is not really there.

9. barplot vs countplot?

`barplot` needs a category and a numerical column, and the bar height is the mean of that numerical column — same as a groupby mean. `countplot` needs only one categorical column and the bar height is the number of rows — same as `value_counts()`. So countplot is just barplot with counting built in.

10. What is the black vertical line on top of a barplot?

It is the error bar — by default the 95% confidence interval of the mean, computed by bootstrapping. A long line means that group has few samples or high variance. I remove it with `errorbar=None`.

11. Histogram vs KDE plot?

A histogram puts values into bins and counts them, so the shape depends on the bin size. A KDE places a small smooth curve on each point and adds them up, giving a smooth density curve whose area is 1. KDE is easier to read and better for comparing groups, but its smoothness depends on bandwidth, and it can show values that do not exist in the data, for example negative values for an always-positive column.

12. What is heatmap vs clustermap?

Heatmap colours a matrix in the given row and column order. Clustermap first performs hierarchical clustering, reorders rows and columns so similar ones come together, and draws dendrograms on the side. Clustermap is figure-level and has built-in scaling options.

13. What is a pairplot and when is it not useful?

pairplot draws every numerical column against every other, with distributions on the diagonal. It is great for a quick look at a small dataset. With many columns it becomes n×n tiny plots, which is slow and unreadable — then I use a correlation heatmap first and pairplot only the selected columns through `vars=`.

14. What does jointplot show?

The relationship between two numerical columns in the centre, plus the individual distribution of each column on the top and right margins. `kind` can be scatter, hex, kde or reg — I use hex for very large data because dots would overlap.

15. How do you check correlation between all features?

`sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm', vmin=-1, vmax=1)`. I use a diverging colormap because zero is a meaningful centre, and I mask the upper triangle because the matrix is symmetric. By default it is Pearson correlation, which catches only linear relations; Spearman is used for monotonic non-linear ones.

▸ C. Practical / code

16. figsize is not working on my seaborn plot. Why?

Because it is a figure-level function — it creates its own figure, so the earlier `plt.figure(figsize=...)` is ignored. I use `height` and `aspect`, where width = height × aspect.

17. How do you put seaborn plots inside subplots?

Create the grid with `fig, ax = plt.subplots(2,2)` and pass `ax=ax[0,0]` to **axes-level** functions. Figure-level functions cannot be put inside subplots.

18. How do you change the title or labels of a seaborn plot?

For axes-level, the function returns an Axes, so `ax = sns.boxplot(...)` then `ax.set_title()`, or simply `plt.title()`. For figure-level, it returns a grid object, so `g.set_axis_labels('x','y')`, `g.set_titles()`, and `g.fig.suptitle()`.

19. What is faceting? How is it different from hue?

Faceting splits data into separate small panels by a category, using `col` and `row`. `hue` keeps everything in one panel with different colours. hue is better for direct comparison; faceting is better when there are many categories or heavy overlap. Facet axes are shared by default, so comparison stays fair.

20. Difference between FacetGrid, PairGrid and JointGrid?

FacetGrid repeats the same plot for each category. PairGrid plots every pair of columns in a matrix. JointGrid shows one bivariate plot with marginal distributions. `catplot` / `relplot` / `displot` are the ready-made versions of FacetGrid, `pairplot` of PairGrid, and `jointplot` of JointGrid.

21. What are the different seaborn styles and contexts?

Five styles — darkgrid, whitegrid, dark, white, ticks — which control the background and grid. Four contexts — paper, notebook, talk, poster — which scale fonts, lines and markers. `sns.set_theme()` sets both at once.

22. What types of palettes are there and when do you use each?

Qualitative for unordered categories like `Set2` or `deep`. Sequential for low-to-high values like `Blues` or `rocket`. Diverging when there is a meaningful centre, like a correlation matrix, for example `coolwarm` or `vlag`. For accessibility I use the `colorblind` palette.

23. What happened to distplot?

It is deprecated and removed in recent versions. Its replacements are `histplot` for the axes-level histogram with optional KDE, `kdeplot` for density only, and `displot` as the figure-level version.

24. What does sns.despine() do?

It removes the top and right border lines (spines) of the plot, which makes the chart look cleaner. In plain Matplotlib I would write `ax.spines['top'].set_visible(False)` for each one.

25. How do you handle a swarmplot warning on large data?

The warning means points could not be placed without overlapping. I either increase the figure width, reduce the marker size, sample the data, or switch to stripplot or a violinplot which handle large data better.

26. Your scatterplot has 1 lakh points and looks like a blob. What do you do?

That is overplotting. I lower `alpha`, or use `jointplot(kind='hex')`, or a 2D `kdeplot`, or `histplot` with both x and y which makes a 2D histogram. Sampling the rows is also an option.

27. How do you show the distribution of a numerical column for each category?

Three ways depending on the need: `boxplot` for a compact summary with outliers, `violinplot` for the full shape, and `histplot` or `kdeplot` with `hue` for overlapping curves. If there are many categories I facet with `displot(col='category')`.

28. In EDA, which seaborn plots do you use and in what order?

First univariate — histplot for numerical columns, countplot for categorical ones, boxplot to find outliers. Then bivariate — scatterplot for two numerical, boxplot or barplot for category vs numerical, and a correlation heatmap. Then multivariate — add hue, facet with col, and pairplot on the important columns.

29. How does Seaborn help in feature selection for a machine learning model?

A correlation heatmap shows which features relate to the target and which features are highly correlated with each other, which signals multicollinearity. Boxplots of a feature grouped by the target class show whether that feature separates the classes. pairplot with `hue=target` shows if classes form separate clusters.

30. Can you use Matplotlib code on a Seaborn plot?

Yes, that is the normal workflow. Axes-level functions return a Matplotlib Axes, so `plt.title()`, `plt.xticks(rotation=45)`, `ax.set_ylim()` and `plt.savefig()` all work. For figure-level plots I access `g.fig` and `g.axes` to do the same.

FINAL TIP

Whenever you name a Seaborn function in an interview, add one line on **why that plot for that data type** and whether it is **figure-level or axes-level**. Those two habits make you sound like someone who has actually used the library.